



Blueprint Technique

Préparé par : Anas Babouih

Entreprise : AgriEdge

Stack : Django (Backend) + Next.js / React (Frontend) + GraphQL (Apollo)

1. Résumé Exécutif

AgriEdge a besoin d'une application digitale pour remplacer un processus de gestion des congés manuel (papier / email) par un système centralisé, traçable et automatisé. L'application couvrira l'ensemble du cycle de vie d'une demande de congé : soumission par l'employé, validation hiérarchique à deux niveaux (Manager N1 puis RH), mise à jour automatique des soldes, notifications, et reporting pour l'équipe Capital Humain.

Objectifs métier :

- Réduire le temps de traitement d'une demande de congé.
- Garantir la traçabilité et l'auditabilité des décisions (conservation 5 ans).
- Automatiser le calcul des jours décomptés (règles marocaines : dimanches non ouvrés, gestion du premier samedi offert, jours fériés).
- Fournir aux RH une vue consolidée des soldes et un export Excel natif.
- Offrir une expérience utilisateur moderne et réactive via une interface découplée.

Utilisateurs cibles : Employés, Managers N1, équipe RH (Capital Humain), Administrateurs système.

Bénéfices attendus : réduction des erreurs de calcul de solde, visibilité en temps réel pour les managers, historique complet pour les audits internes, et une base technique évolutive (API GraphQL) pour de futures intégrations (paie, SIRH, mobile).

2. Choix d'Architecture : Backend Django + Frontend Découplé

Le stack retenu combine deux besoins exprimés dans le cadrage :

Couche	Technologie	Justification
Backend / Logique métier	Django (Python)	Admin intégré pour les RH, ORM robuste pour les calculs de dates, sécurité native (CSRF, injection SQL), auth par rôles (Groups & Permissions)
API	GraphQL (Graphene-Django)	Requêtes flexibles depuis le frontend (éviter l'over-fetching), un seul endpoint, typage fort du schéma
Frontend	Next.js / React	Rendu hybride (SSR/CSR), routing basé fichiers, bonnes performances SEO/first-load pour un portail RH interne
Client GraphQL	Apollo Client	Cache normalisé, gestion d'état serveur, souscriptions temps réel pour les notifications
Visualisation de données	ECharts (Apache)	Graphiques de reporting RH (soldes, tendances d'absences)
Style	Tailwind CSS	Cohérence avec la maquette prévue, développement rapide, responsive natif
Base de données	PostgreSQL / MySQL	Compatible Django ORM, transactions fiables pour les soldes
Tâches de fond	Celery + Redis (ou Django-Q)	Rappels J-3, emails asynchrones, exports programmés

Note d'architecture : Django sert ici de backend API pur (via GraphQL), et non de moteur de rendu de templates HTML — sauf pour l'interface d'administration RH qui reste sur le Django Admin natif, réservée en interne.

3. Modules Fonctionnels

3.1 Authentification & Rôles

- Connexion (JWT via GraphQL mutation login)
- Gestion de session, rafraîchissement de token
- 4 rôles : Employé, Manager N1, RH, Administrateur

3.2 Espace Employé

- Tableau de bord : solde actuel, historique, statut des demandes en cours
- Création d'une demande de congé avec calcul instantané des jours (AJAX/GraphQL en direct)
- Upload de pièce jointe (justificatif)
- Annulation d'une demande (si statut le permet)

3.3 Workflow d'Approbation

- Vue "À valider" pour Manager N1 et RH
- Actions : Approuver / Refuser / Commenter
- Rappels automatiques après 3 jours d'inactivité (J-3)

3.4 Gestion RH (Capital Humain)

- Gestion des soldes, profils, départements
- Gestion des jours fériés (PublicHoliday)
- Export Excel des soldes (openpyxl)
- Vue consolidée / calendrier d'équipe

3.5 Notifications

- Emails transactionnels (soumission, décision, rappel)
- Notifications in-app (souscriptions GraphQL/WebSocket)

3.6 Reporting & Analytics

- Graphiques ECharts : taux d'absentéisme, répartition par type de congé, tendances par département
- Export des rapports

3.7 Administration

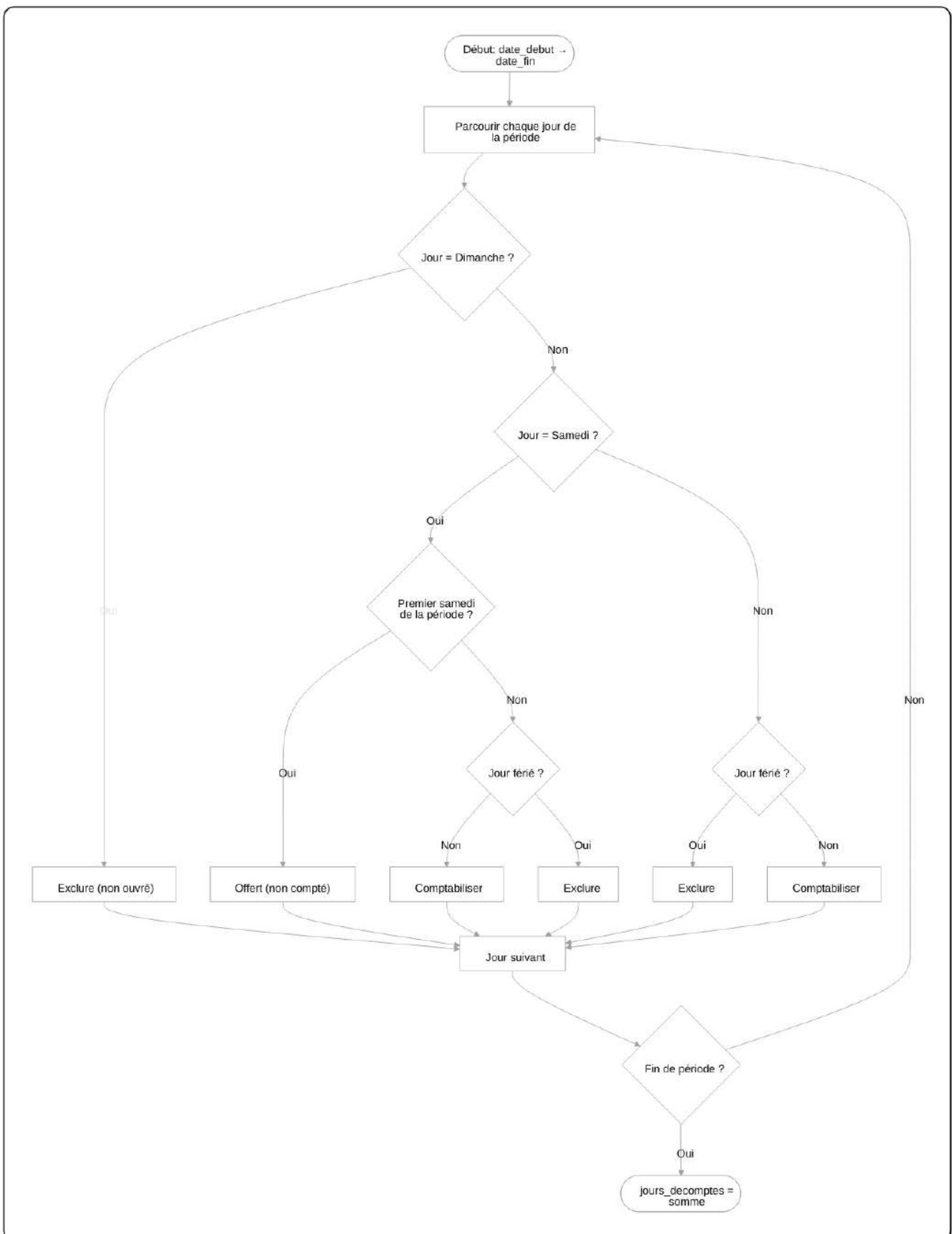
- Paramétrage des types de congé, règles de calcul, jours fériés
- Gestion des rôles et permissions

4. Exigences Non-Fonctionnelles

Catégorie	Exigence
Performance	Calcul des jours décomptés instantané côté client via appel GraphQL léger
Sécurité	Protection CSRF/XSS/SQLi native Django ; JWT signé ; validation stricte des entrées côté schéma GraphQL
Scalabilité	Séparation frontend/backend permettant un scaling indépendant (CDN pour Next.js, réplicas pour Django)
Disponibilité	Cible 99.5% ; supervision des workers Celery
Accessibilité	Interface responsive Tailwind, conforme WCAG AA de base
Traçabilité	Logs d'audit conservés 5 ans (changements de statut, décisions)
Confidentialité	Données RH sensibles chiffrées au repos ; accès RBAC strict
Maintenabilité	Schéma GraphQL versionné, séparation claire des couches (services, resolvers, modèles)

5. Logique Métier Centrale (Core Logic)

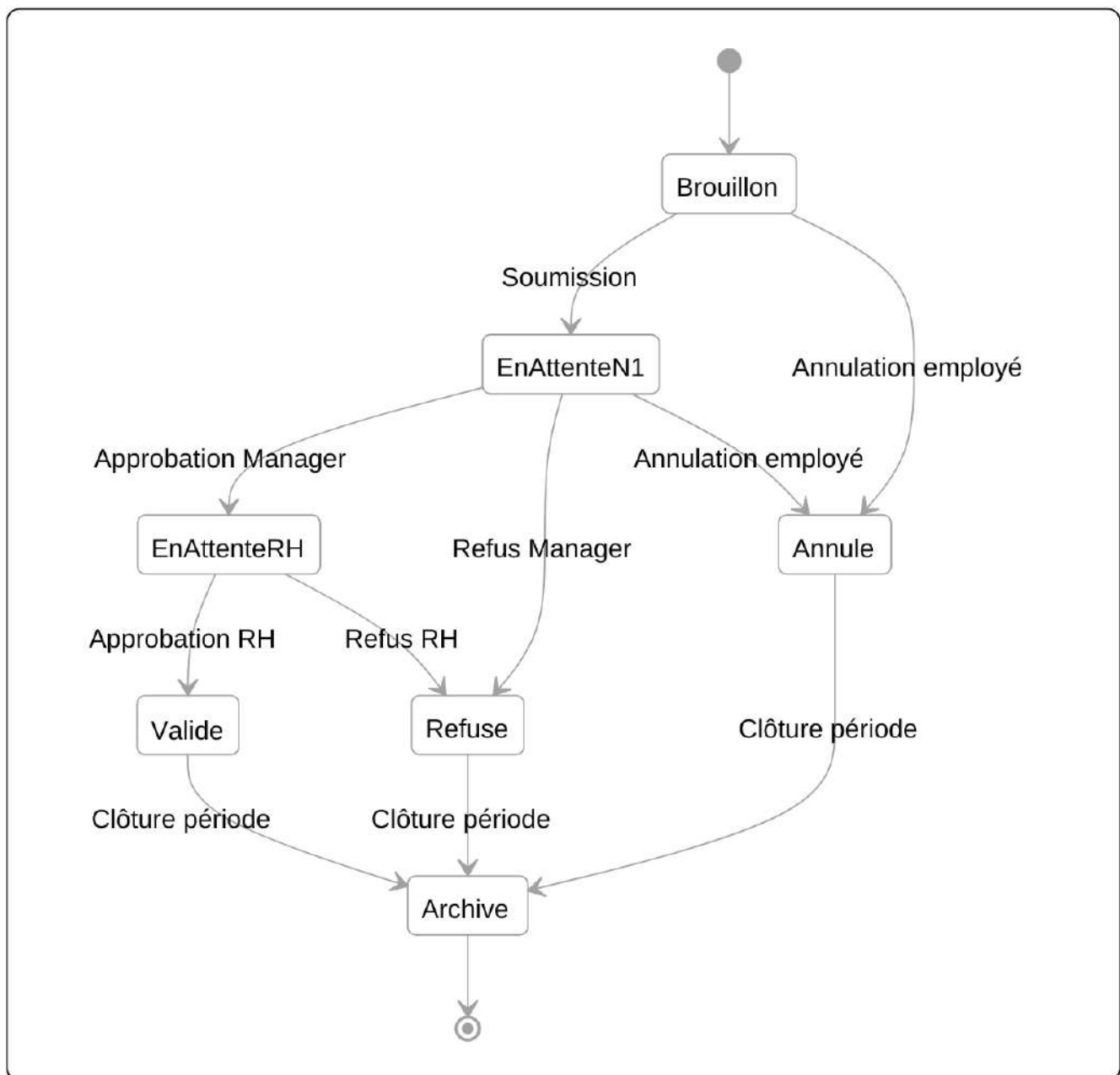
5.1 Algorithme de Calcul des Jours Décomptés



Règles :

- Dimanches toujours exclus (non ouvrés).
- Le premier samedi rencontré dans la demande est offert (non décompté) ; à partir du deuxième samedi, les samedis sont comptabilisés.
- Tout jour férié (semaine ou samedi) est soustrait, via consultation de `PublicHoliday`.
- Le calcul s'exécute côté backend (source de vérité) et est exposé via une requête GraphQL légère pour un aperçu instantané côté frontend avant soumission.

5.2 Machine à États — Workflow de Validation



Étape	Statut	Droits d'édition
1	Brouillon	Employé : édition/suppression totale
2	En attente N1	Lecture seule employé ; action Manager N1
3	En attente RH	Lecture seule ; action RH
4	Validé / Refusé	Verrouillé ; solde mis à jour si validé
5	Archivé	Lecture seule, conservé pour audit (5 ans)

Chaque transition de statut déclenche un Django Signal (`post_save`) qui : (1) notifie l'acteur suivant par email, (2) journalise l'action dans `AuditLog`, (3) republie l'événement via une souscription GraphQL pour mise à jour temps réel côté frontend.

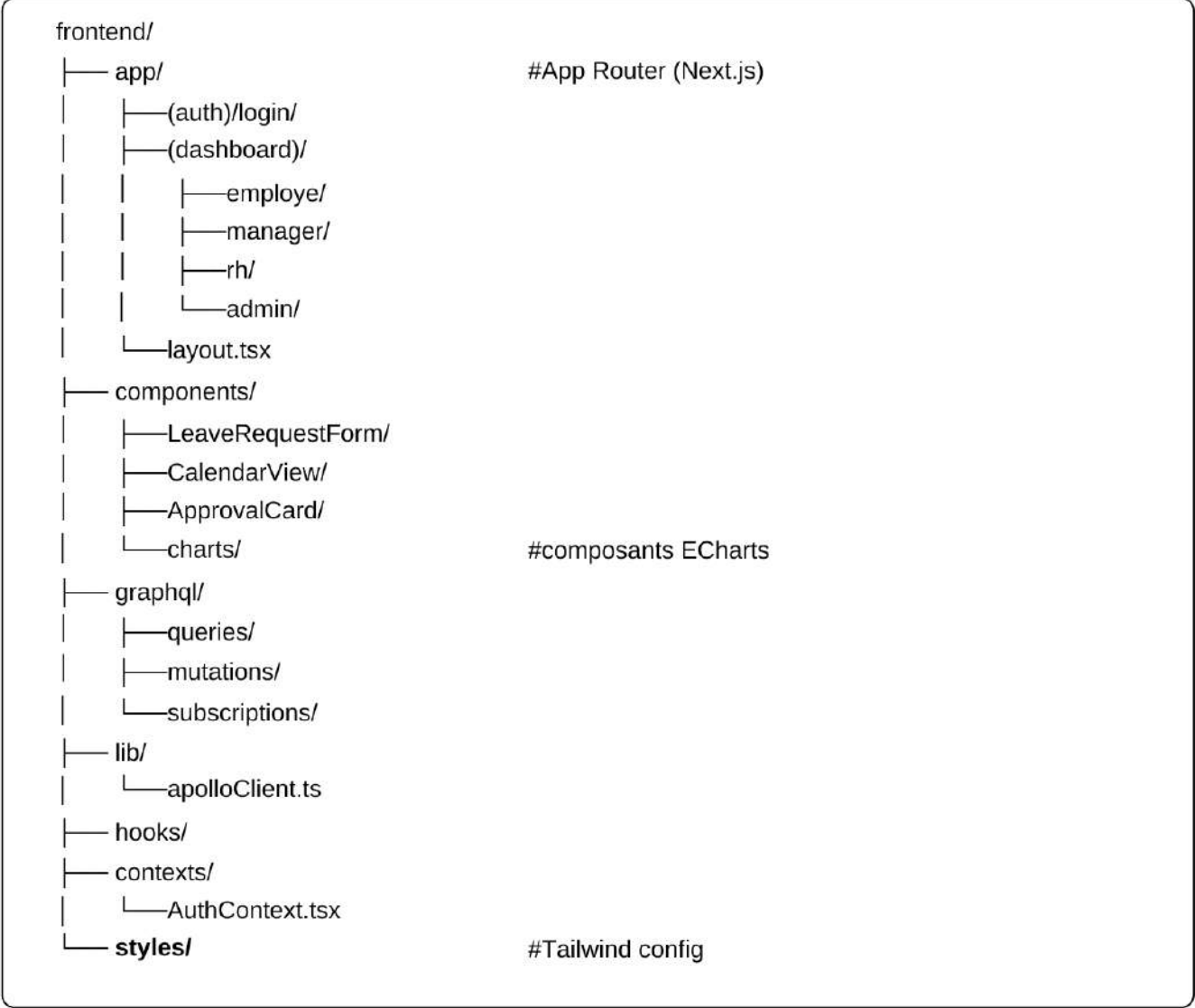
4.3 Tâches de Fond

- Rappel J-3 : tâche Celery quotidienne qui scanne les demandes En attente N1 / En attente RH sans décision depuis 3 jours et relance l'approbateur.
- Emails transactionnels : envoyés de manière asynchrone via Celery pour ne pas bloquer les mutations GraphQL.
- Export Excel : génération à la demande via `openpyxl`, déclenchée par une mutation GraphQL, retournant un lien de téléchargement signé.

5. Rôles Utilisateurs

Rôle	Permissions
Employé	Créer/éditer une demande en Brouillon, consulter son solde et son historique, annuler avant validation N1
Manager N1	Consulter les demandes de son équipe, approuver/refuser au niveau 1, consulter le calendrier d'équipe
RH	Approuver/refuser au niveau 2, gérer les soldes/profils/jours fériés, exporter les rapports, accès Django Admin
Administrateur	Gestion des rôles, paramétrage système, supervision des logs d'audit, configuration des types de congé

6. Structure du Projet Frontend (Next.js)



7. Structure du Projet Backend (Django)

backend/


```

├── config/                                #settings, urls, schema racine
├── apps/
│   ├── employees/
│   ├── leaves/
│   │   ├── models.py
│   │   ├── schema.py                    #types & resolvers GraphQL
│   │   ├── services.py                 #algorithme de calcul, workflow
│   │   ├── signals.py
│   │   └── tasks.py                     #tâches Celery
│   ├── holidays/
│   └── notifications/
├── core/
│   ├── permissions.py
│   └── audit.py
└── celery.py

```

8. Sécurité

- Authentification : JWT (via django-graphql-jwt ou équivalent), refresh tokens. Autorisation :
- contrôle par rôle au niveau des resolvers GraphQL (pas seulement côté frontend).
- Protection native Django : CSRF, échappement XSS, ORM paramétré (anti-injection SQL).
- Validation des entrées : au niveau du schéma GraphQL (types stricts) et des serializers/services Django.
- Upload de fichiers : restriction des types MIME, taille max, stockage isolé (hors webroot).
- Audit : journalisation systématique des changements de statut dans AuditLog, conservation 5 ans.
- Rate limiting : throttling sur les mutations sensibles (login, soumission).

9. Déploiement

- Conteneurisation Docker pour chaque service (frontend, backend, worker, DB, cache).
- Nginx comme reverse proxy et point d'entrée TLS.
- CI/CD : build/test automatisés, déploiement par environnement (dev/staging/prod).
- Variables d'environnement centralisées (.env par service, secrets hors dépôt).

10. Feuille de Route (Roadmap)

Phase	Contenu
Phase 1	Authentification, modèles de base (Employee, Department)
Phase 2	Module congés : création, calcul des jours, brouillon
Phase 3	Workflow d'approbation N1 → RH, notifications
Phase 4	Reporting RH, export Excel, calendrier d'équipe
Phase 5	Souscriptions temps réel, tableaux de bord ECharts, optimisations
